Documentation Station Météo ATMOS 41 Gen 2

Projet

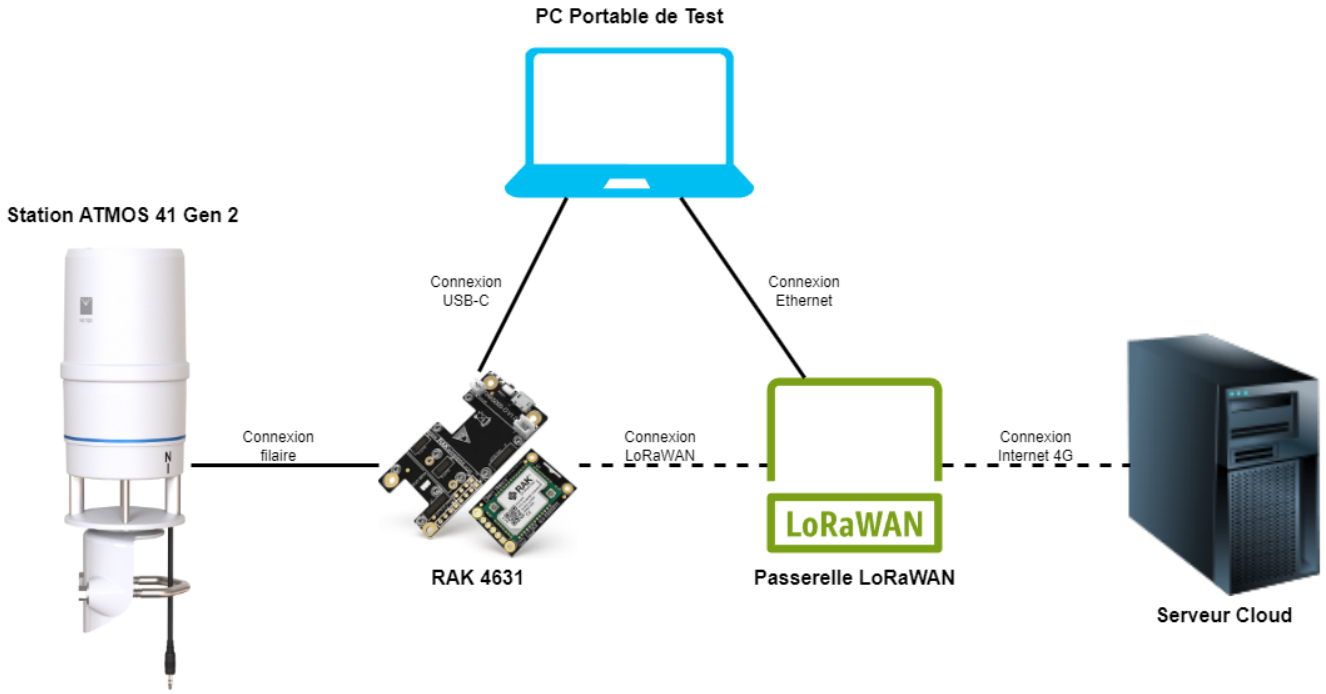
1. Contexte et introduction

1.1 Contexte du projet

Ce projet s'inscrit dans **Terra Forma**, un programme national porté par le CNRS (EquipEx+) et financé par l'ANR, qui vise à construire un observatoire de l'Anthropocène à partir de capteurs environnementaux ouverts, frugaux et déployables sur le terrain. L'objectif est d'obtenir des mesures fiables à haute résolution spatiale et temporelle pour mieux comprendre les évolutions environnementales et appuyer la prise de décision.

Dans ce cadre, notre travail consiste à développer une chaîne complète autour d'une station météo **ATMOS 41 Gen 2** : acquisition des mesures, traitement local sur carte **RAK**, transmission via **LoRaWAN**, puis réception/décodage côté serveur cloud (**TTN**). L'enjeu principal est de concilier robustesse de transmission, faible consommation énergétique et exploitation simple des données.

1.2 Schéma explicatif



Ce schéma résume l'architecture testée : la station produit les mesures, la carte RAK les prépare, la passerelle les relaie vers le cloud, puis le serveur les décode et les met à disposition pour l'analyse.

2. Branchements physiques

2.1 RAK + STATION + PC

Version de la connectique de notre station météo ATMOS 41 Gen 2 :

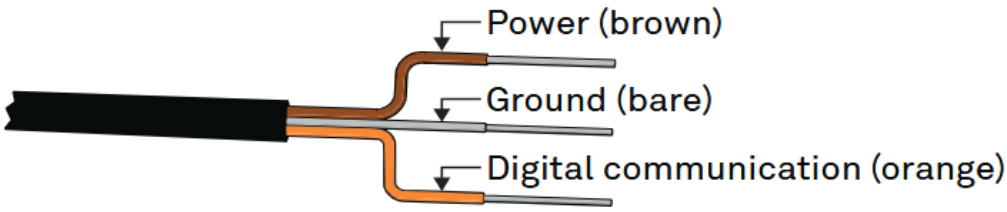
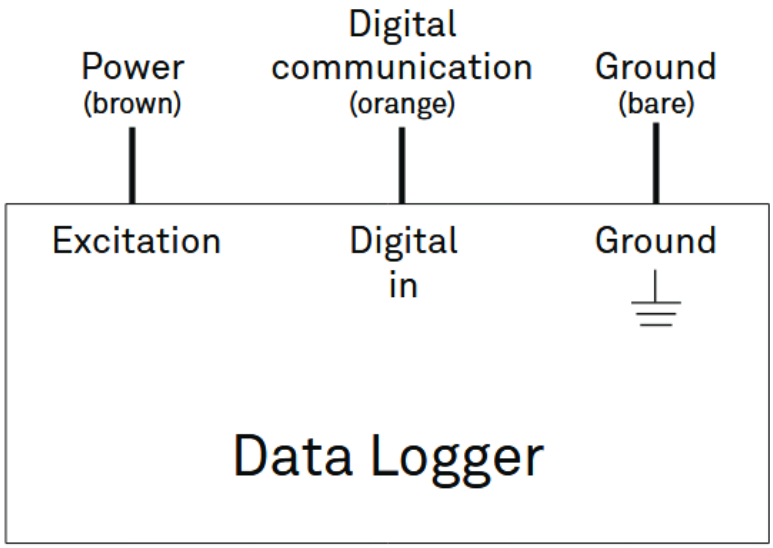
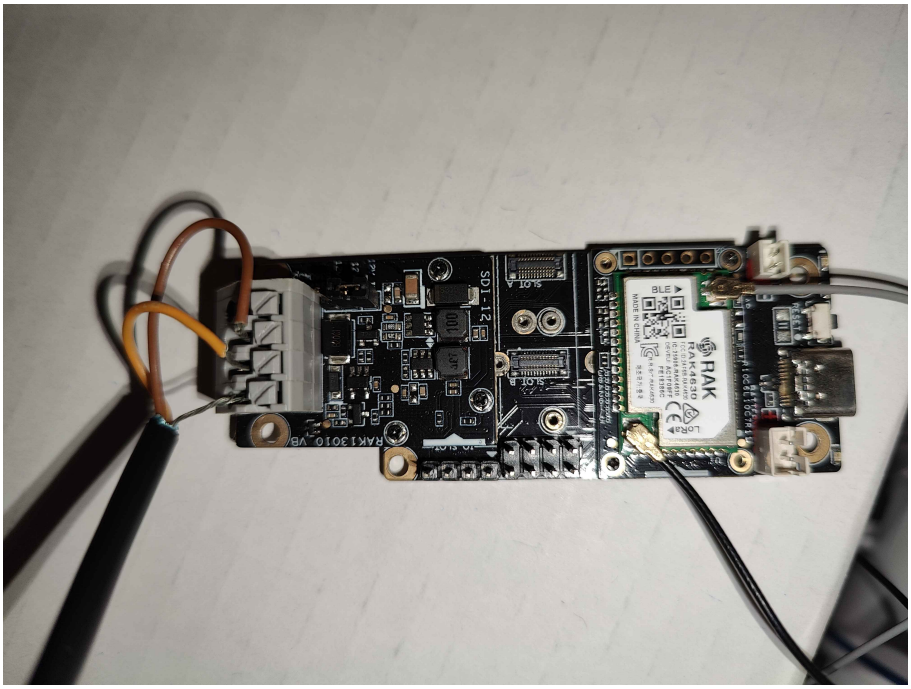


Figure 2 Pigtail wiring



La carte RAK est connectée à la station via ses câbles dénudés :



Connectivité avec la carte RAK :

Pendant la phase de test la carte RAK est reliée via câble USB-C à un ordinateur portable. La connectivité avec l'ordinateur portable permet l'alimentation de la carte RAK ainsi que les différents capteurs de la station ATMOS.

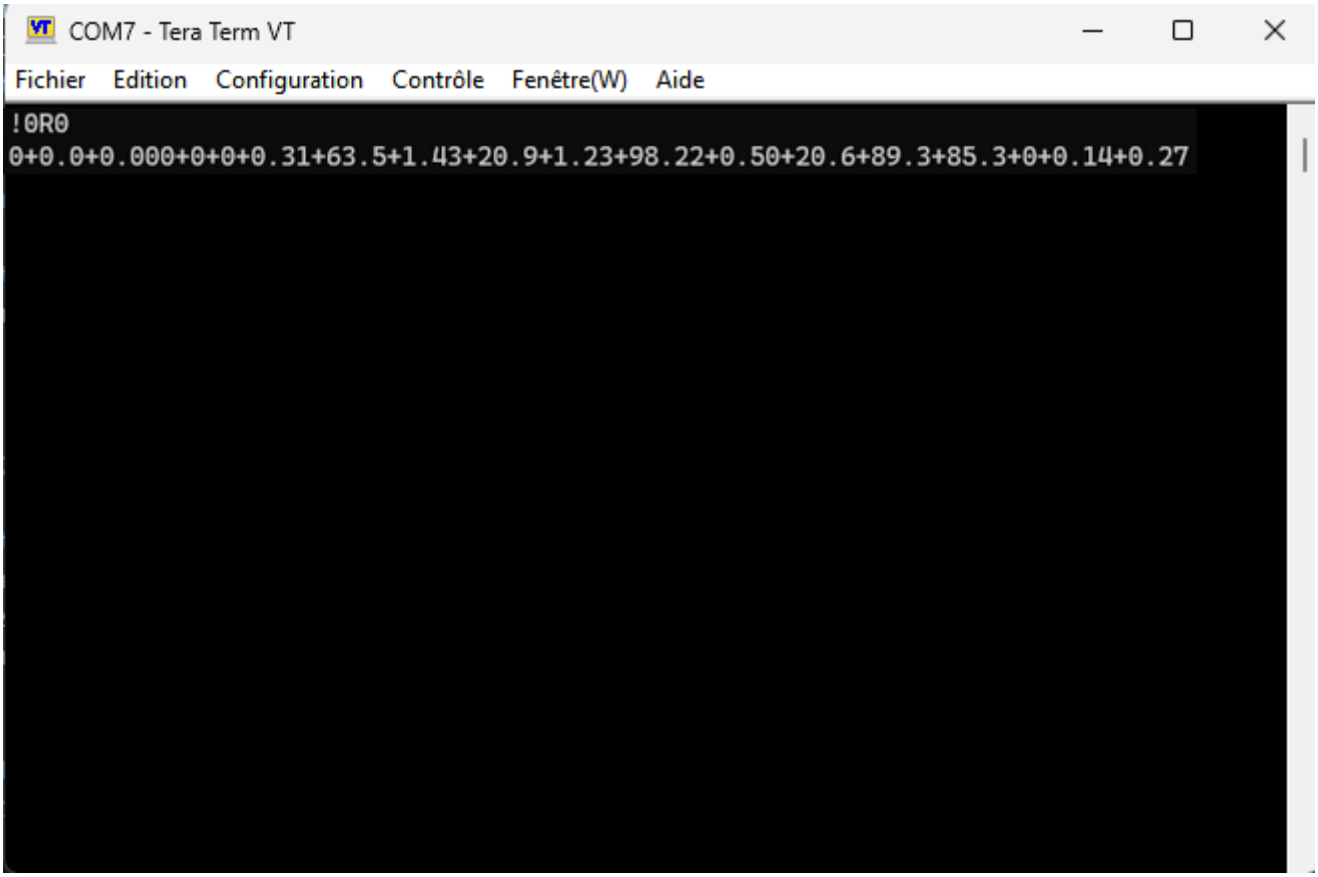
2.2 PASSERELLE + PC Ethernet

Pendant la phase de test, la connectivité avec la passerelle pour l'administrer se fait via câble ethernet entre le PC Portable et la Passerelle LoRaWAN. La passerelle se connecte au réseau internet avec une carte SIM 4G permettant de passer par le réseau mobile pour atteindre la serveur cloud où sont stockées les données.

3. Logiciels utilisés

3.1 TeraTerm

Pour des premières connexions avec la station météo et le PC Portable, on peut utiliser TeraTerm pour envoyer des commandes et dialoguer avec :



Si le firmware utilisé est RUI3 et pas nRF vous pouvez utiliser des commandes AT pour récupérer des informations sur le matériel :

```
    AT
    OK
    AT+VER=?    (Commande pour voir la version du firmware installé)
    AT+VER=RUI_4.2.1_RAK4631
    OK
    AT+NWM=?   (Commande pour vérifier le mode de fonctionnement - 0:LoRa P2P,
1:LoRaWAN)
    AT+NWM=1
```

```
    OK
    AT+NJM=?    (Commande pour voir le mode de jonction - 0:ABP, 1:OTAA)
    AT+NJM=1
    OK
    AT+DEVEUI=? (Commande pour lire le DEVEUI)
    AT+DEVEUI=AC1F09FFFE183B9B
    OK
    AT+APPEUI=? (Commande pour lire l'AppEUI)
    AT+APPEUI=0000000000000000
    OK
    AT+APPKEY=? (Commande pour lire la clé AppKey)
    AT+APPKEY=00000000000000000000000000000000
    OK
    AT+NJS=? (Commande pour vérifier si la connexion LoRaWAN est en cours - 0:Non,
1:Oui)
```

3.2 nrfUtil (mise à jour des firmwares)

Documentation de la mise à jour RAK en nRF :

```
https://docs.rakwireless.com/Product-Categories/WisBlock/RAK4631-
R/DFU/#converting-rak4631-r-to-rak4631
```

3.3 Arduino UNO

Pour l'écriture des scripts à envoyer à la station on utilise Arduino UNO. Cela permet d'à la fois sélectionner le port du PC connecté à la carte RAK ainsi que de coder et d'observer les réponses reçues de la station.

3.4 Navigateur internet pour accès au serveur Cloud

Utilisation libre mais testé avec Google Chrome et Mozilla Firefox.

4. Installation du RAK

4.1 Version firmware recommandée

Suivre la documentation (3.2 nrfUtil (mise à jour des firmwares))

5. Récupération des valeurs avec TeraTerm

5.1 Commandes `0M0!`, `0D0!`, `0R0!`

Un grand nombre de commandes permet de récupérer les mesures de la station. La commande `0M0!` permet de réaliser une nouvelle mesure. La commande `0D0!` permet de récupérer les data des nouvelles mesures. La commande `0R0!` permet de relire les dernières données que l'on a mesuré.

Mais on s'est rendu compte que ses commandes ne permettaient pas de récupérer toutes les mesures. En creusant un peu plus on a trouvé la commande `0R0!` qui permet elle de récupérer toutes les mesures de la

station :

```
!0R0
0+0.0+0.000+0+0+0.31+63.5+1.43+20.9+1.23+98.22+0.50+20.6+89.3+85.3+0+0.14+0.27
```

6. Code de récupération des données sur la station (v1)

Le but de cette première version de code est de vérifier l'intégralité des données récoltables du côté de la station. A ce niveau là, la seule connectique est entre la station météo, la carte RAK et le PC Portable pour visualiser les résultats. Le code est simple, on envoi juste la commande `0R0!` puis on la travaille pour l'afficher dans un format lisible pour un humain.

Code :

```
/**
    @file SecondAtmosCode.ino
*/

#include "RAK13010_SDI12.h"

#define TX_PIN   WB_IO6
#define RX_PIN   WB_IO5
#define OE       WB_IO4

#define SENSOR_ADDRESS '0'

RAK_SDI12 mySDI12(RX_PIN, TX_PIN, OE);

String sdiResponse = "";
String sdiMsgStr   = "";
bool   sdiMsgReady = false;

void setup()
{
    pinMode(WB_IO2, OUTPUT);
    digitalWrite(WB_IO2, HIGH);

    Serial.begin(115200);
    time_t timeout = millis();
    while (!Serial && (millis() - timeout) < 5000) delay(100);

    Serial.println("Opening SDI-12 bus.");
    mySDI12.begin();
    delay(500);

    mySDI12.forceListen();
    Serial.println("Press ENTER to send 0R0!");
}

void loop()
{
    static uint8_t serialMsgRflag = 0;
```

```
    // Detect ENTER key
    if (Serial.available())
    {
        char c = Serial.read();
        if (c == '\n' || c == '\r')
            serialMsgRflag = 1;
    }

    // Read SDI-12 incoming
    int avail = mySDI12.available();
    if (avail < 0)
    {
        mySDI12.clearBuffer();
    }
    else if (avail > 0)
    {
        for (int i = 0; i < avail; i++)
        {
            char c = mySDI12.read();
            if (c == '\n')
                sdiMsgReady = true;
            else
                sdiMsgStr += String(c);
        }
    }

    // When full SDI-12 message received
    if (sdiMsgReady)
    {
        Serial.println("\nRaw SDI-12:");
        Serial.println(sdiMsgStr);

        decodeR0(sdiMsgStr);

        sdiMsgStr = "";
        sdiMsgReady = false;
        serialMsgRflag = 0;
    }

    // User requested a measurement
    if (serialMsgRflag == 1)
    {
        serialMsgRflag = 2;
        String cmd = String(SENSOR_ADDRESS) + "R0!";
        mySDI12.sendCommand(cmd);
        Serial.println("T: " + cmd);
    }
}

/**
 * Decode and print the values from the R0 response
 */
void decodeR0(String msg)
{
```

```
    // Remove address character (e.g., "0")
    if (msg.length() < 2) return;
    msg.remove(0, 1);

    // Split using '+'
    float values[20];
    int vCount = 0;

    int start = 0;
    for (int i = 0; i <= msg.length(); i++)
    {
        if (msg[i] == '+' || msg[i] == '-' || i == msg.length())
        {
            if (i != start)
            {
                String token = msg.substring(start, i);
                values[vCount++] = token.toFloat();
            }
            start = i;
        }
    }

    Serial.println("\n------ Decoded ATMOS-41 (R0) ------");
    if (vCount < 17)
    {
        Serial.println("Error: Too few values!");
        return;
    }

    Serial.print("Rain (raw) ................. "); Serial.println(values[0]);
    Serial.print("Rain (corr) ................ "); Serial.println(values[1]);
    Serial.print("Rain flag 1 ................ "); Serial.println(values[2]);
    Serial.print("Rain flag 2 ................ "); Serial.println(values[3]);
    Serial.print("Rain last min (mm) ......... "); Serial.println(values[4]);
    Serial.print("Wind direction (°) ......... "); Serial.println(values[5]);
    Serial.print("Wind speed avg (m/s) ....... "); Serial.println(values[6]);
    Serial.print("Wind gust (m/s) ............ "); Serial.println(values[7]);
    Serial.print("Vapor pressure (kPa) ....... "); Serial.println(values[8]);
    Serial.print("Humidity (%) ............... "); Serial.println(values[9]);
    Serial.print("Rain rate (mm/h) ........... "); Serial.println(values[10]);
    Serial.print("Air temp (°C) .............. "); Serial.println(values[11]);
    Serial.print("Solar radiation (W/m²) ..... "); Serial.println(values[12]);
    Serial.print("Barometric pressure (kPa) .. "); Serial.println(values[13]);
    Serial.print("Wind avg 5 min (m/s) ....... "); Serial.println(values[14]);
    Serial.print("Wind U component (m/s) ..... "); Serial.println(values[15]);
    Serial.print("Wind V component (m/s) ..... "); Serial.println(values[16]);

    Serial.println("----------------------------------\n");
}
```

7. Visualisation des données (v1)

Exemple de récolte et visualisation des données :

```
------ Decoded ATMOS-41 (R0) ------
Rain (raw) ................... 0.00
Rain (corr) ................. 0.00
Rain flag 1 ................. 0.00
Rain flag 2 ................. 0.00
Rain last min (mm) ......... 0.24
Wind direction (°) ......... 237.70
Wind speed avg (m/s) ....... 1.93
Wind gust (m/s) ............ 22.30
Vapor pressure (kPa) ....... 1.30
Humidity (%) ............... 100.91
Rain rate (mm/h) ........... 0.48
Air temp (°C) .............. 22.60
Solar radiation (W/m²) ..... 89.20
Barometric pressure (kPa) .. 85.00
Wind avg 5 min (m/s) ....... 0.00
Wind U component (m/s) ..... -0.13
Wind V component (m/s) ..... -0.20
-----------------------------------
```

8. Test complet de récupération de données (v2)

8.1 Choix de la nouvelle méthode de récupération et transformation des données

La version v2 de notre script est la suivante : Le but de cette version est de récupérer 6 valeurs sur les 17 disponibles de la station, les transformer pour un transports LoRaWAN et du stockage sur un serveur cloud.

```
1) Récupération des données (commande capteur)
    La commande est construite avec l'adresse capteur + R0! : String cmd =
String(SENSOR_ADDRESS) + "R0!";, puis envoyée par "mySDI12.sendCommand(cmd)".
    Le cycle est déclenché par le timer "startMeasurementCycle=true" puis traité
dans loop().

2) Séparation / parsing de la trame
    La lecture se fait caractère par caractère via mySDI12.read() tant que
mySDI12.available() > 0.
    On détecte la fin de trame avec '\n' (sdiMsgReady = true) ou '\r' ignoré, le
reste est concaténé dans "sdiMsgStr".
    "decodeATMOS41()" enlève d'abord l'adresse (msg.remove(0,1)), puis découpe
selon +, espace, et - pour reconstruire les valeurs numériques.
    Ensuite, on map vers currentMeasure avec les indices ATMOS: pluie [1], vitesse
du vent [4], direction du vent [5], température [7], pression [9], humidité [10].

3) Envoi en paquet LoRaWAN
    send_lora_frame() vérifie d'abord que la jonction LoRaWAN est faite
(lmh_join_status_get() == LMH_SET).
    Les float sont convertis en entiers (×100) pour garder 2 décimales. Les
```

```
paquets LoRa n'acceptant pas trop les float dans les envois de données.
    Les champs sont packés en binaire dans cet ordre: pluie, direction du vent, vitesse du vent, température, humidité, pression.
    La taille finale de notre paquet LoRaWAN est de 12 octets, avec chaque mesures récoltés faisant 2 octets.
```

9. Configuration côté RAK

9.1 Fichier `.ino`

Code :

```
#include <Arduino.h>
#include <LoRaWan-Arduino.h>
#include <SPI.h>
#include "RAK13010_SDI12.h"

// --- Définition des broches et constantes ---
// Broches SDI-12 (slot A par défaut)
#define TX_PIN   WB_IO6
#define RX_PIN   WB_IO5
#define OE       WB_IO4
#define POWER_PIN WB_IO2
#define SENSOR_ADDRESS '0'

// Configuration LoRaWAN
#define SCHED_MAX_EVENT_DATA_SIZE APP_TIMER_SCHED_EVENT_DATA_SIZE
#define SCHED_QUEUE_SIZE 60
#define LORAWAN_APP_DATA_BUFF_SIZE 64
#define LORAWAN_APP_TX_DUTYCYCLE   30000   // 30 seconds
#define JOINREQ_NBTRIALS 3

// --- Objets globaux et variables ---
RAK_SDI12 mySDI12(RX_PIN, TX_PIN, OE);

// Objets LoRaWAN
hw_config hwConfig;
TimerEvent_t appTimer;
static uint8_t m_lora_app_data_buffer[LORAWAN_APP_DATA_BUFF_SIZE];
static lmh_app_data_t m_lora_app_data = {m_lora_app_data_buffer, 0, 0, 0, 0};

// Drapeaux de la machine d'états
volatile bool startMeasurementCycle = false; // Set by Timer, read by Loop
bool sdiMsgReady = false;
String sdiMsgStr = "";

// Structure de données capteur -> LoRa
struct WeatherData {
    float rain_mm = 0.0;
    float wind_dir_deg = 0.0;
    float wind_speed_ms = 0.0;
```

```
    float temp_c = 0.0;
    float humidity_pct = 0.0;
    float pressure_kpa = 0.0;
} currentMeasure;

// --- Clés LoRaWAN (OTAA) ---
uint8_t nodeDeviceEUI[8] = {0x70, 0xB3, 0xD5, 0x7E, 0xD0, 0x07, 0x45, 0xE6};
uint8_t nodeAppEUI[8]    = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
uint8_t nodeAppKey[16]   = {0x14, 0x31, 0x4F, 0xD7, 0x96, 0x11, 0xBB, 0x5F, 0xB5,
0xD5, 0x01, 0xAE, 0xF1, 0xEF, 0x62, 0xEF};

uint32_t nodeDevAddr = 0x260116F8;
uint8_t nodeNwsKey[16] = {0};
uint8_t nodeAppsKey[16] = {0};

// --- Déclarations anticipées ---
static void lorawan_has_joined_handler(void);
static void lorawan_rx_handler(lmh_app_data_t *app_data);
static void lorawan_confirm_class_handler(DeviceClass_t Class);
static void lorawan_join_failed_handler(void);
static void tx_lora_periodic_handler(void);
static void send_lora_frame(void);
void decodeATMOS41(String msg);

static lmh_callback_t lora_callbacks = {
  BoardGetBatteryLevel, BoardGetUniqueId, BoardGetRandomSeed,
  lorawan_rx_handler, lorawan_has_joined_handler,
  lorawan_confirm_class_handler, lorawan_join_failed_handler
};

static lmh_param_t lora_param_init = {
  LORAWAN_ADR_ON, LORAWAN_DEFAULT_DATARATE, LORAWAN_PUBLIC_NETWORK,
  JOINREQ_NBTRIALS, LORAWAN_DEFAULT_TX_POWER, LORAWAN_DUTYCYCLE_OFF
};

// --- Fonctions utilitaires ---
static void put_u16_be(uint8_t *buf, uint8_t &i, uint16_t v) {
  buf[i++] = (uint8_t)(v >> 8);
  buf[i++] = (uint8_t)(v & 0xFF);
}
static void put_i16_be(uint8_t *buf, uint8_t &i, int16_t v) {
  uint16_t uv = (uint16_t)v;
  buf[i++] = (uint8_t)(uv >> 8);
  buf[i++] = (uint8_t)(uv & 0xFF);
}

// --- Initialisation ---
void setup() {
    // Initialisation du port série
    Serial.begin(115200);
    time_t timeout = millis();
    while (!Serial && (millis() - timeout) < 5000) delay(100);
    Serial.println("====================================");
    Serial.println("   METEO STATION: ATMOS 41 + LORA    ");
```

```
    Serial.println("====================================");

    // Initialisation matérielle SDI-12
    pinMode(POWER_PIN, OUTPUT);
    digitalWrite(POWER_PIN, HIGH); // Alimente le bus SDI-12

    Serial.println("[SDI12] Opening bus...");
    mySDI12.begin();
    delay(500);
    mySDI12.forceListen();
    sdiMsgStr.reserve(200);

    // Initialisation matérielle LoRa
#ifdef RAK4630
    lora_rak4630_init();
#else
    lora_hardware_init(hwConfig);
#endif

    // Initialisation de la pile LoRaWAN
    // Configuration du timer
    appTimer.timerNum = 3;
    TimerInit(&appTimer, tx_lora_periodic_handler);

    lmh_setDevEui(nodeDeviceEUI);
    lmh_setAppEui(nodeAppEUI);
    lmh_setAppKey(nodeAppKey);
    lmh_setNwkSKey(nodeNwsKey);
    lmh_setAppSKey(nodeAppsKey);
    lmh_setDevAddr(nodeDevAddr);

    lmh_init(&lora_callbacks, lora_param_init, true, CLASS_A,
LORAMAC_REGION_EU868);

    Serial.println("[LORA] Sending Join Request...");
    lmh_join();
}

// --- Boucle principale ---
void loop() {
    // État 1 : déclenchement de la mesure (via le timer)
    if (startMeasurementCycle) {
        startMeasurementCycle = false; // Réinitialise le drapeau

        Serial.println("\n--- [CYCLE START] ---");

        // 1) Nettoyage des buffers précédents
        sdiMsgStr = "";
        sdiMsgReady = false;
        while(mySDI12.available()) mySDI12.read();

        // 2) Envoi de la commande
        String cmd = String(SENSOR_ADDRESS) + "R0!";
        mySDI12.sendCommand(cmd);
```

```
        Serial.println("[SDI12] Transmitted: " + cmd);

        // Pas d'attente bloquante ici : la lecture continue dans la suite de la
boucle.
    }

    // État 2 : lecture non bloquante de la réponse capteur
    int avail = mySDI12.available();
    if (avail > 0) {
        for (int i = 0; i < avail; i++) {
            char c = mySDI12.read();
            if (c == '\n') {
                sdiMsgReady = true;
            } else if (c != '\r') {
                sdiMsgStr += String(c);
            }
        }
    }

    // État 3 : décodage et envoi LoRa
    if (sdiMsgReady) {
        Serial.println("[SDI12] Raw: " + sdiMsgStr);

        // 1) Parse de la trame dans la structure currentMeasure
        decodeATMOS41(sdiMsgStr);

        // 2) Envoi du paquet via LoRa
        send_lora_frame();

        // 3) Nettoyage
        sdiMsgStr = "";
        sdiMsgReady = false;
        Serial.println("--- [CYCLE END] ---\n");
    }
}

// --- Fonctions métier ---

// Parseur ATMOS41
void decodeATMOS41(String msg) {
    if (msg.length() < 2) return;
    msg.remove(0, 1); // Supprime l'adresse capteur ('0')

    float values[25];
    int vCount = 0;
    String currentVal = "";

    // Logique de parsing
    for (int i = 0; i < msg.length(); i++) {
        char c = msg[i];
        bool isDelimiter = (c == '+' || c == ' ');
        bool isSign = (c == '-');

        if ((isDelimiter || isSign) && currentVal.length() > 0) {
```

```
            if (vCount < 25) values[vCount++] = currentVal.toFloat();
            currentVal = "";
            if (isSign) currentVal += c;
        } else if (c != '+' && c != ' ') {
            currentVal += c;
        }
    }
    if (currentVal.length() > 0 && vCount < 25) values[vCount++] =
currentVal.toFloat();

    // Mapping vers la structure globale (indices du manuel ATMOS 41 Gen 2)
    // [1] Precip, [4] WindSpd, [5] WindDir, [7] Temp, [9] Baro, [10] RH
    if(vCount > 10) {
        currentMeasure.rain_mm       = values[1];
        currentMeasure.wind_speed_ms = values[4];
        currentMeasure.wind_dir_deg  = values[5];
        currentMeasure.temp_c        = values[7];
        currentMeasure.pressure_kpa  = values[9];
        currentMeasure.humidity_pct  = values[10];

        Serial.printf("[DATA] Temp: %.2f C, Hum: %.2f %%, Rain: %.2f mm\n",
                      currentMeasure.temp_c, currentMeasure.humidity_pct,
currentMeasure.rain_mm);
    } else {
        Serial.println("[ERR] Parsing failed or incomplete frame.");
    }
}

// Affichage du buffer en HEX
void debugBufferState(uint8_t* buf, int len) {
    Serial.print("   -> Buffer actuel (" + String(len) + " octets) : [ ");
    for (int j = 0; j < len; j++) {
        if (buf[j] < 0x10) Serial.print("0"); // Ajout d'un 0 pour l'alignement
(ex: 0F)
        Serial.print(buf[j], HEX);
        Serial.print(" ");
    }
    Serial.println("]");
}

// Envoi LoRa
static void send_lora_frame(void) {
    if (lmh_join_status_get() != LMH_SET) {
        Serial.println("[LORA] Not joined yet. Skip send.");
        return;
    }

    // Conversion des flottants en entiers (x100 pour 2 décimales)
    int16_t  rain_cnt      = (int16_t)(currentMeasure.rain_mm * 100);
    uint16_t wind_dir      = (uint16_t)(currentMeasure.wind_dir_deg);
    int16_t  wind_spd_cnt  = (int16_t)(currentMeasure.wind_speed_ms * 100);
    int16_t  temp_cnt      = (int16_t)(currentMeasure.temp_c * 100);
    int16_t  hum_cnt       = (int16_t)(currentMeasure.humidity_pct * 100);
    int16_t  press_cnt     = (int16_t)(currentMeasure.pressure_kpa * 100);
```

```
    uint8_t i = 0;
    m_lora_app_data.port = LORAWAN_APP_PORT;

    Serial.println("--- Début de la construction du payload ---");

    // 1) Rain
    Serial.print("Ajout Rain (Int16): "); Serial.println(rain_cnt);
    put_i16_be(m_lora_app_data.buffer, i, rain_cnt);
    debugBufferState(m_lora_app_data.buffer, i);

    // 2) Wind Direction
    Serial.print("Ajout Wind Dir (Uint16): "); Serial.println(wind_dir);
    put_u16_be(m_lora_app_data.buffer, i, wind_dir);
    debugBufferState(m_lora_app_data.buffer, i);

    // 3) Wind Speed
    Serial.print("Ajout Wind Spd (Int16): "); Serial.println(wind_spd_cnt);
    put_i16_be(m_lora_app_data.buffer, i, wind_spd_cnt);
    debugBufferState(m_lora_app_data.buffer, i);

    // 4) Temperature
    Serial.print("Ajout Temp (Int16): "); Serial.println(temp_cnt);
    put_i16_be(m_lora_app_data.buffer, i, temp_cnt);
    debugBufferState(m_lora_app_data.buffer, i);

    // 5) Humidity
    Serial.print("Ajout Hum (Int16): "); Serial.println(hum_cnt);
    put_i16_be(m_lora_app_data.buffer, i, hum_cnt);
    debugBufferState(m_lora_app_data.buffer, i);

    // 6) Pressure
    Serial.print("Ajout Press (Int16): "); Serial.println(press_cnt);
    put_i16_be(m_lora_app_data.buffer, i, press_cnt);
    debugBufferState(m_lora_app_data.buffer, i);

    Serial.println("--- Fin du payload ---");


    m_lora_app_data.buffsize = i;

    lmh_error_status error = lmh_send(&m_lora_app_data, LMH_UNCONFIRMED_MSG);
    if (error == LMH_SUCCESS) {
        Serial.println("[LORA] Packet sent successfully.");
    } else {
        Serial.printf("[LORA] Send failed. Error: %d\n", error);
    }
}

// --- Gestionnaires d'événements ---
// Le timer ne fait que lever un drapeau, sans traitement lourd.
static void tx_lora_periodic_handler(void) {
    TimerSetValue(&appTimer, LORAWAN_APP_TX_DUTYCYCLE);
    TimerStart(&appTimer);
```

```
        // Déclenche la boucle principale
        startMeasurementCycle = true;
    }

    static void lorawan_has_joined_handler(void) {
        Serial.println("[LORA] Joined!");
        lmh_class_request(CLASS_A);
        // Démarre le timer dès la jonction
        TimerSetValue(&appTimer, LORAWAN_APP_TX_DUTYCYCLE);
        TimerStart(&appTimer);
        // Déclenche une première mesure immédiate
        startMeasurementCycle = true;
    }

    static void lorawan_join_failed_handler(void) {
        Serial.println("[LORA] Join Failed. Retrying...");
        lmh_join(); // Nouvelle tentative
    }

    static void lorawan_rx_handler(lmh_app_data_t *app_data) {
        // Optionnel : traiter le downlink
    }
    static void lorawan_confirm_class_handler(DeviceClass_t Class) {}
```

9.2 Output de la station (v2)

```
[SDI12] Opening bus...
[LORA] Sending Join Request...
[LORA] Joined!

--- [CYCLE START] ---
[SDI12] Transmitted: 0R0!
[SDI12] Raw: 0+0.0+0.000+0+0+0.31+63.5+1.43+20.9+1.23+98.22+0.50+20.6+89.3+85.3+0+0.14+0.27
[DATA] Temp: 20.90 C, Hum: 0.50 %, Rain: 0.00 mm
[LORA] Packet sent successfully.
--- [CYCLE END] ---


--- [CYCLE START] ---
[SDI12] Transmitted: 0R0!
[SDI12] Raw: 0+0.0+0.000+0+0+0.10+84.0+0.18+21.2+1.23+98.22+0.49+20.7+89.2+85.0+0+0.01+0.10
[DATA] Temp: 21.20 C, Hum: 0.49 %, Rain: 0.00 mm
[LORA] Packet sent successfully.
--- [CYCLE END] ---
```

10. Configuration côté passerelle

10.1 SSH sur la passerelle

10.1.1 Fichier `activate_cell-fichierActivation4GPasserelle.sh`

```
#!/bin/sh
echo "Stopping existing connections..."
sudo killall -9 pppd chat 2>/dev/null
sudo rm -f /var/lock/LCK..modem_at0

echo "Resetting Radio Power..."
sudo /usr/sbin/mts-io-sysfs store radio-power 0
sleep 2
sudo /usr/sbin/mts-io-sysfs store radio-power 1
sleep 5

echo "Dialing Lebara (APN: internet)..."
sudo pppd call gsm

echo "Waiting for IP address..."
sleep 15
ip addr show ppp0
```

10.1.2 Configuration de la passerelle pour passer via 4G pour atteindre le réseau internet

En SSH sur la passerelle via câble ethernet connecté entre le PC Portable et la passerelle, il faut charger le fichier suivant : `activate_cell.sh`

```
#!/bin/sh
echo "Stopping existing connections..."
sudo killall -9 pppd chat 2>/dev/null
sudo rm -f /var/lock/LCK..modem_at0

echo "Resetting Radio Power..."
sudo /usr/sbin/mts-io-sysfs store radio-power 0
sleep 2
sudo /usr/sbin/mts-io-sysfs store radio-power 1
sleep 5

echo "Dialing Lebara (APN: internet)..."
sudo pppd call gsm

echo "Waiting for IP address..."
sleep 15
ip addr show ppp0
```

10.1.3 Fichier de configuration LoRa

Après avoir accès au réseau internet, il faut configurer la connectivité LoRaWAN de la passerelle. Il faut alors charger en SSH le fichier de configuration LoRa : `/var/config/lora/global_conf.json`

```
{
	"SX1301_conf": {
		"lorawan_public": true,
		"clksrc": 0,
		"lbt_cfg": {
			"enable": false,
			"rssi_target": 160,
			"nb_channel": 1,
			"start_freq": 869525000,
			"scan_time_us": 5000,
			"tx_delay_1ch_us": 4000000,
			"tx_delay_2ch_us": 4000000
		},
		"antenna_gain": 0,
		"radio_0": {
			"enable": true,
			"type": "SX1257",
			"freq": 867500000,
			"rssi_offset": -162,
			"tx_enable": true,
			"tx_freq_min": 863000000,
			"tx_freq_max": 870000000
		},
		"radio_1": {
			"enable": true,
			"type": "SX1257",
			"freq": 868500000,
			"rssi_offset": -162,
			"tx_enable": false
		},
		"chan_multiSF_0": {
			"enable": true,
			"radio": 1,
			"if": -400000
		},
		"chan_multiSF_1": {
			"enable": true,
			"radio": 1,
			"if": -200000
		},
		"chan_multiSF_2": {
			"enable": true,
			"radio": 1,
			"if": 0
		},
		"chan_multiSF_3": {
			"enable": true,
			"radio": 0,
			"if": -400000
		},
		"chan_multiSF_4": {
			"enable": true,
			"radio": 0,
```

```
                "if": -200000
            },
            "chan_multiSF_5": {
                "enable": true,
                "radio": 0,
                "if": 0
            },
            "chan_multiSF_6": {
                "enable": true,
                "radio": 0,
                "if": 200000
            },
            "chan_multiSF_7": {
                "enable": true,
                "radio": 0,
                "if": 400000
            },
            "chan_Lora_std": {
                "enable": true,
                "radio": 1,
                "if": -200000,
                "bandwidth": 250000,
                "spread_factor": 7
            },
            "chan_FSK": {
                "enable": true,
                "radio": 1,
                "if": 300000,
                "bandwidth": 125000,
                "datarate": 50000
            },
                "tx_lut_0":{"dig_gain":1,"mix_gain":10,"pa_gain":1,"rf_power":4},
                "tx_lut_1":{"dig_gain":3,"mix_gain":8,"pa_gain":3,"rf_power":11},
                "tx_lut_2":{"dig_gain":2,"mix_gain":11,"pa_gain":2,"rf_power":12},
                "tx_lut_3":{"dig_gain":0,"mix_gain":10,"pa_gain":2,"rf_power":13},
                "tx_lut_4":{"dig_gain":3,"mix_gain":14,"pa_gain":2,"rf_power":14},
                "tx_lut_5":{"dig_gain":1,"mix_gain":12,"pa_gain":2,"rf_power":15},
                "tx_lut_6":{"dig_gain":0,"mix_gain":12,"pa_gain":2,"rf_power":16},
                "tx_lut_7":{"dig_gain":2,"mix_gain":14,"pa_gain":2,"rf_power":17},
                "tx_lut_8":{"dig_gain":2,"mix_gain":15,"pa_gain":2,"rf_power":18},
                "tx_lut_9":{"dig_gain":1,"mix_gain":15,"pa_gain":2,"rf_power":19},
                "tx_lut_10":{"dig_gain":0,"mix_gain":9,"pa_gain":3,"rf_power":20},
                "tx_lut_11":
{"dig_gain":1,"mix_gain":10,"pa_gain":3,"rf_power":21},
                "tx_lut_12":
{"dig_gain":0,"mix_gain":10,"pa_gain":3,"rf_power":22},
                "tx_lut_13":
{"dig_gain":0,"mix_gain":11,"pa_gain":3,"rf_power":23},
                "tx_lut_14":
{"dig_gain":2,"mix_gain":14,"pa_gain":3,"rf_power":24},
                "tx_lut_15":{"dig_gain":0,"mix_gain":15,"pa_gain":3,"rf_power":25}
        },
        "gateway_conf": {
            "gateway_ID": "008000000001103B",
```

```
            "server_address": "eu1.cloud.thethings.network",
            "serv_port_up": 1700,
            "serv_port_down": 1700,
            "keepalive_interval": 10,
            "stat_interval": 30,
            "push_timeout_ms": 100,
            "forward_crc_valid": true,
            "forward_crc_error": true,
            "forward_crc_disabled": false,
          "synch_word": 52,
                  "autoquit_threshold": 60
      }
  }
```

10.1.4 Redémarrage du module LoRaWAN

Toujours en connexion SSH sur la machine : `sudo /etc/init.d/lora-packet-forwarder restart`

11. Configuration côté serveur

11.1 Fonction Decode (voir `FonctionDecode.txt`)

Côté serveur, à la réception des données il faut une fonction de décodage afin de récupérer les valeurs qui ont été modifiées pour être envoyées via LoRaWAN.

```
function decodeUplink(input) {
              var bytes = input.bytes;
              var data = {};

              // Helper: Read Signed Int16 (Big Endian)
              function readInt16BE(idx) {
                var val = (bytes[idx] << 8) | bytes[idx + 1];
                // Handle negative values (Two's Complement)
                if (val > 0x7FFF) {
                  val -= 0x10000;
                }
                return val;
              }

              // Helper: Read Unsigned Uint16 (Big Endian)
              function readUint16BE(idx) {
                return (bytes[idx] << 8) | bytes[idx + 1];
              }

              // --- Decoding & Scaling ---

              // 1. Rain (mm)
              // C++: rain_mm * 100 -> Decoder: / 100
              data.rain_mm = readInt16BE(0) / 100;
```

```
                // 2. Wind Direction (Degrees)
                // C++: wind_dir_deg (No scaling)
                data.wind_dir_deg = readUint16BE(2);

                // 3. Wind Speed (m/s)
                // C++: wind_speed_ms * 100 -> Decoder: / 100
                data.wind_speed_ms = readInt16BE(4) / 100;

                // 4. Temperature (°C)
                // C++: temp_c * 100 -> Decoder: / 100
                data.temperature_c = readInt16BE(6) / 100;

                // 5. Humidity (%)
                // C++: humidity_pct * 100 -> Decoder: / 100
                data.humidity_pct = readInt16BE(8) / 100;

                // 6. Pressure (kPa)
                // C++: pressure_kpa * 100 -> Decoder: / 100
                // Note: Output is in kPa. (e.g., 97.46 kPa).
                // If you want hPa, multiply this result by 10.
                data.pressure_kpa = readInt16BE(10) / 10; // Wait, correct math
below
                // Logic: raw 9746 / 100 = 97.46 kPa.
                data.pressure_kpa = readInt16BE(10) / 100;

                return {
                  data: data,
                  warnings: [],
                  errors: []
                };
              }
```

11.2 Fichiers JSON récupérés (ou CSV)

Depuis le serveur cloud on peut générer un fichier `.csv` ou `.json` regroupant toutes les données sur une certaine durée. Exemple de fichier récupéré :


```
{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260BEC0C"},"received_at":"2026-02-
05T08:57:46.354555223Z","uplink_message":
{"f_port":2,"frm_payload":"AAAAxwAmCHAANCYV","decoded_payload":
{"message":"\\u0000\\u0000\\u0000Ç\\u0000&\bp\\u00004&\u0015"},"rx_metadata":
[{"gateway_ids":{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":62457860,"rssi":-14,"chann
el_rssi":-14,"snr":8.8,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"channel_index":2,"received_at":"2026-02-
05T08:57:46.099956837Z"}],"settings":{"data_rate":{"lora":{"bandwidth":125000,
"spreading_factor":9, "coding_rate":"4/5"}}, "frequency":"868500000",
"timestamp":62457860},"received_at":"2026-02-
```

```
05T08:57:46.148389532Z","consumed_airtime":"0.205824s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260B5189"},"received_at":"2026-02-
05T09:18:27.626014263Z","uplink_message":
{"f_port":2,"frm_payload":"AAAA4wAtCRAAMyYS","decoded_payload":
{"message":"\\u0000\\u0000\\u0000ã\\u0000-\t\u0010\\u00003&\u0012"},"rx_metadata":
[{"gateway_ids":{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":1303727180,"rssi":-26,"cha
nnel_rssi":-26,"snr":11.8,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"received_at":"2026-02-05T09:18:27.383875916Z"}],"settings":{"data_rate":
{"lora":{"bandwidth":125000, "spreading_factor":9, "coding_rate":"4/5"}},
"frequency":"868100000", "timestamp":1303727180},"received_at":"2026-02-
05T09:18:27.417408610Z","consumed_airtime":"0.205824s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260BE3DA"},"received_at":"2026-02-
05T09:28:33.909938646Z","uplink_message":
{"f_port":2,"frm_payload":"AAAA3gAfCPwANCYS","decoded_payload":
{"message":"\\u0000\\u0000\\u0000Þ\\u0000\u001f\bü\\u00004&\u0012"},"rx_metadata":
[{"gateway_ids":{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":1910005220,"rssi":-25,"cha
nnel_rssi":-25,"snr":12.2,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"received_at":"2026-02-05T09:28:33.675355809Z"}],"settings":{"data_rate":
{"lora":{"bandwidth":125000, "spreading_factor":9, "coding_rate":"4/5"}},
"frequency":"868100000", "timestamp":1910005220},"received_at":"2026-02-
05T09:28:33.706464461Z","consumed_airtime":"0.205824s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260B9E95"},"received_at":"2026-02-
05T09:34:50.016563717Z","uplink_message":
{"f_port":2,"frm_payload":"AAABOQALCPIAMyYS","decoded_payload":
{"humidity_pct":0.51, "pressure_kpa":97.46, "rain_mm":0, "temperature_c":22.9,
"wind_dir_deg":313, "wind_speed_ms":0.11},"rx_metadata":[{"gateway_ids":
{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":2286102868,"rssi":-17,"cha
nnel_rssi":-17,"snr":11.2,"location":
```

```
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"channel_index":4,"received_at":"2026-02-
05T09:34:49.776493051Z"}],"settings":{"data_rate":{"lora":{"bandwidth":125000,
"spreading_factor":9, "coding_rate":"4/5"}}, "frequency":"867300000",
"timestamp":2286102868},"received_at":"2026-02-
05T09:34:49.807289632Z","consumed_airtime":"0.205824s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260BC60B"},"received_at":"2026-02-
05T09:38:55.284510056Z","uplink_message":
{"f_port":2,"frm_payload":"AAABZAAGCRAANCYS","decoded_payload":
{"humidity_pct":0.52, "pressure_kpa":97.46, "rain_mm":0, "temperature_c":23.2,
"wind_dir_deg":356, "wind_speed_ms":0.06},"rx_metadata":[{"gateway_ids":
{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":2531360532,"rssi":-21,"cha
nnel_rssi":-21,"snr":12.2,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"channel_index":3,"received_at":"2026-02-
05T09:38:55.045372043Z"}],"settings":{"data_rate":{"lora":{"bandwidth":125000,
"spreading_factor":9, "coding_rate":"4/5"}}, "frequency":"867100000",
"timestamp":2531360532},"received_at":"2026-02-
05T09:38:55.076571698Z","consumed_airtime":"0.205824s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260BC60B"},"received_at":"2026-02-
05T09:39:25.140210751Z","uplink_message":
{"f_port":2,"f_cnt":1,"frm_payload":"AAABIwA3CUIAMyYS","decoded_payload":
{"humidity_pct":0.51, "pressure_kpa":97.46, "rain_mm":0, "temperature_c":23.7,
"wind_dir_deg":291, "wind_speed_ms":0.55},"rx_metadata":[{"gateway_ids":
{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":2561226507,"rssi":-25,"cha
nnel_rssi":-25,"snr":9.5,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"channel_index":1,"received_at":"2026-02-
05T09:39:24.898199163Z"}],"settings":{"data_rate":{"lora":{"bandwidth":125000,
"spreading_factor":7, "coding_rate":"4/5"}}, "frequency":"868300000",
"timestamp":2561226507},"received_at":"2026-02-
05T09:39:24.928511523Z","consumed_airtime":"0.071936s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
```

```
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260BC60B"},"received_at":"2026-02-
05T09:39:55.110013291Z","uplink_message":
{"f_port":2,"f_cnt":2,"frm_payload":"AAABKQASCVYAMiYS","decoded_payload":
{"humidity_pct":0.5, "pressure_kpa":97.46, "rain_mm":0, "temperature_c":23.9,
"wind_dir_deg":297, "wind_speed_ms":0.18},"rx_metadata":[{"gateway_ids":
{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":2591199411,"rssi":-23,"cha
nnel_rssi":-23,"snr":8,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"channel_index":2,"received_at":"2026-02-
05T09:39:54.876411814Z"}],"settings":{"data_rate":{"lora":{"bandwidth":125000,
"spreading_factor":7, "coding_rate":"4/5"}}, "frequency":"868500000",
"timestamp":2591199411},"received_at":"2026-02-
05T09:39:54.906913765Z","consumed_airtime":"0.061696s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260BC60B"},"received_at":"2026-02-
05T09:40:25.111873806Z","uplink_message":
{"f_port":2,"f_cnt":3,"frm_payload":"AAABRwAUCUIAMiYS","decoded_payload":
{"humidity_pct":0.5, "pressure_kpa":97.46, "rain_mm":0, "temperature_c":23.7,
"wind_dir_deg":327, "wind_speed_ms":0.2},"rx_metadata":[{"gateway_ids":
{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":2621215796,"rssi":-23,"cha
nnel_rssi":-23,"snr":10,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"channel_index":6,"received_at":"2026-02-05T09:40:24.878435Z"}],"settings":
{"data_rate":{"lora":{"bandwidth":125000, "spreading_factor":7,
"coding_rate":"4/5"}}, "frequency":"867700000",
"timestamp":2621215796},"received_at":"2026-02-
05T09:40:24.908751451Z","consumed_airtime":"0.061696s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}

{"result":{"end_device_ids":{"device_id":"my-rak","application_ids":
{"application_id":"rak4631-test-
01"},"dev_eui":"70B3D57ED00745E6","dev_addr":"260BC60B"},"received_at":"2026-02-
05T09:40:55.114853271Z","uplink_message":
{"f_port":2,"f_cnt":4,"frm_payload":"AAABMQATCRAAMyYS","decoded_payload":
{"humidity_pct":0.51, "pressure_kpa":97.46, "rain_mm":0, "temperature_c":23.2,
"wind_dir_deg":305, "wind_speed_ms":0.19},"rx_metadata":[{"gateway_ids":
{"gateway_id":"eui-
008000000001103b","eui":"008000000001103B"},"timestamp":2651215596,"rssi":-24,"cha
nnel_rssi":-24,"snr":9.8,"location":
{"latitude":48.1198379104281,"longitude":-1.62957584419057,"source":"SOURCE_REGIST
RY"},"channel_index":7,"received_at":"2026-02-
05T09:40:54.878122170Z"}],"settings":{"data_rate":{"lora":{"bandwidth":125000,
```

```
"spreading_factor":7, "coding_rate":"4/5"}}, "frequency":"867900000",
"timestamp":2651215596},"received_at":"2026-02-
05T09:40:54.908938293Z","consumed_airtime":"0.061696s","version_ids":
{"brand_id":"rakwireless", "model_id":"wisblock-4631", "hardware_version":"1.0",
"firmware_version":"1.2.0", "band_id":"EU_863_870"},"network_ids":
{"net_id":"000013","ns_id":"EC656E0000000181","tenant_id":"ttn","cluster_id":"eu1"
,"cluster_address":"eu1.cloud.thethings.network"}}}}
```


11.3 URL du serveur Cloud

eu1.cloud.thethings.network

11.4 Accès à la Gateway et au serveur Cloud

Pour avoir accès au serveur cloud il faut avoir un compte avec login/mot de passe. Pour demander les accès, vous pouvez vous tourner vers Baptiste Gaultier (voir partie 12. Info & Contacts)

12. Info & Contacts

12.1 Contacts côté CNRS

```
- Nicolas Lavenant - nicolas.lavenant@univ-rennes.fr
- Julien Moureau - julien.moureau@univ-rennes.fr
```

12.2 Contacts côté école

Référents : - Baptiste Gaultier - baptiste.gaultier@imt-atlantique.fr - Laurent Toutain - laurent.toutain@imt-atlantique.fr

Etudiants ayant travaillés sur le projet : - Yoann MERLET - yoann.merlet@imt-atlantique.net - Abderrafia El-ghalam - abderrafia.el-ghalam@imt-atlantique.net - Alexis Pompeu - alexis.pompeu@imt-atlantique.net - Mathis Da-cunha - mathis.da-cunha@imt-atlantique.net